Lezione 1; Fondamenti delle Applicazioni Web

Applicazioni distribuite e Architettura Client-Server

Le applicazioni Distribuite

Un'applicazione distribuita è:

- **un sistema in cui i vari componenti che la compongono non si trovano tutti sulla stessa macchina, ma sono distribuiti su più computer collegati in rete.**
  Questi componenti, pur essendo fisicamente separati, **collaborano tra loro** scambiandosi dati e messaggi.

Per organizzare questa collaborazione, le applicazioni distribuite vengono generalmente progettate secondo un'**architettura multilivello (multi-layer)**.

- **Ogni layer (o livello) è formato da un insieme di componenti.**
- **Ogni insieme di componenti ha un preciso ruolo funzionale nel sistema.**

Architettura multilivello

Il modello multilivello più diffuso è l'**architettura a tre livelli (three-tier)**, che suddivide l'applicazione in:

1. **Client Layer (Presentazione)**
   - **È il livello che interagisce direttamente con l'utente.**
   - Mostra le informazioni a schermo e raccoglie i dati inseriti dall'utente.
   - Può essere implementato come:
     - **un'applicazione web in un browser (HTML, CSS, JavaScript),**
     - **un'app desktop,**
     - **oppure una GUI (interfaccia grafica).**

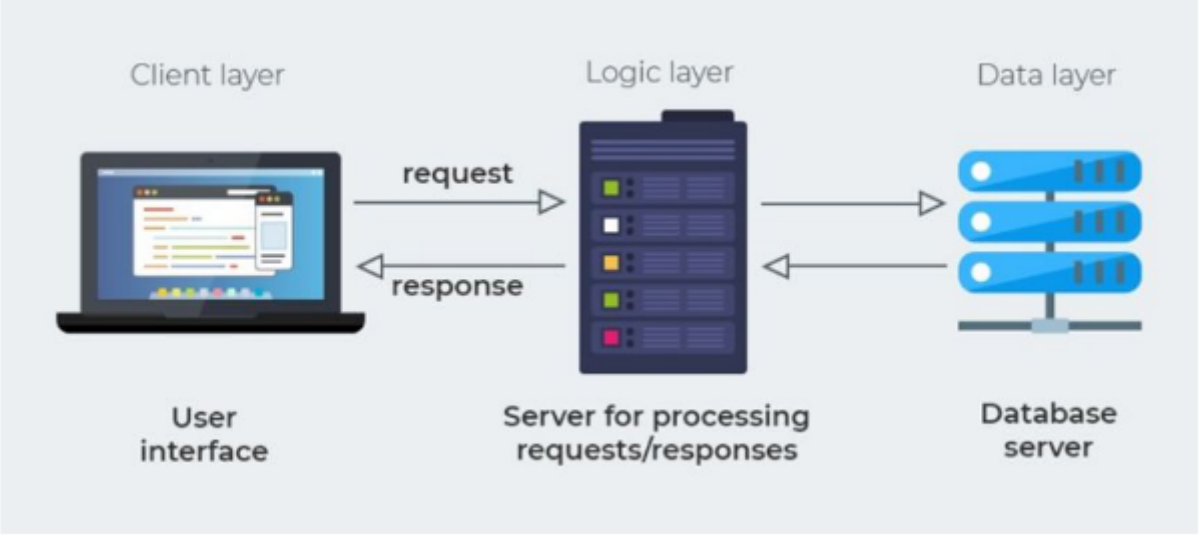
> 🎟 **È il livello superiore: il punto di contatto tra l'utente e il sistema.**

2. **Logic Layer (Applicativo o Business Logic)**
   - **Si occupa di ricevere le richieste dal Client, elaborarle e collegarle con i dati presenti nel livello successivo.**

- Contiene la **logica di business**:
  - **cioè le regole, i controlli e i comportamenti specifici che il sistema deve rispettare.**
- Può **aggiungere, eliminare o modificare dati** nel Data Layer.
- È sviluppato in linguaggi come **Python**, **Java**, **PHP, Perl, Ruby**, e comunica con il database tramite **API o librerie dedicate**.

> 🎟 **È il livello intermedio, il "cervello" del sistema.**

3. **Data Layer (Persistenza dei dati)**
   - **Ha il compito di gestire e archiviare in modo permanente i dati dell'applicazione.**
   - Interagisce soltanto con il Logic Layer (non direttamente con l'utente).
   - Può essere realizzato con database:
     - **Relazionali**: **PostgreSQL, MySQL, MariaDB, Oracle, SQL Server…**
     - **NoSQL**: MongoDB, Cassandra, CouchDB…



architettura a 3 livelli.png

Architettura Client- Server

L'architettura **Client-Server** è il modello fondamentale su cui si basano quasi tutte le applicazioni web moderne.
Descrive una relazione ben precisa:

- **Un programma, il Client, richiede un servizio o una risorsa.**

- **Un altro programma, il Server, riceve la richiesta, la elabora e restituisce la risposta.**

Analogia con il ristorante

Per comprendere meglio come questa architettura lavora prendiamo ad esempio un ristorante e la gestione dei ruoli tra cliente-cameriere-chef:

- **Cliente (Client) → Sei tu seduto al tavolo**. **Hai bisogno di qualcosa (cibo o bevande) e fai una richiesta al cameriere (API)**.
  - Nel web, il client è tipicamente il **browser** sul tuo computer o smartphone.
- **Cucina (Server)** → È sempre attiva, pronta a ricevere ordini. Quando arriva una richiesta, la cucina **elabora** l'ordine (prepara il piatto) e restituisce il risultato (il piatto pronto).
  - Nel web, il server è un **computer potente** connesso a Internet, che ospita l'applicazione e i dati.

Flusso di comunicazione

Esattamente come nel ristorante anche qui il processo di interazione segue sempre gli stessi passaggi:

1. **Request (Richiesta)** → **Il client invia una richiesta al server.**
   (Esempio: "Dammi la homepage di questo sito").
2. **Processing (Elaborazione)** → **Il server riceve la richiesta e la elabora.**
   (Esempio: consulta un database, esegue un calcolo, prepara la risposta).
3. **Response (Risposta)** → **Il server invia la risposta al client.**
   (Esempio: la pagina HTML, un file JSON, o un messaggio di errore).

👉 **Questo modello centralizza la logica e i dati sul server, mantenendo il client leggero e focalizzato solo sulla presentazione.**

Regole generali del modello Client-Server:

1. **Il Server eroga servizi su richiesta.**
2. **I Client si collegano al server e chiedono risorse o servizi.**
3. **Entrambi devono usare un protocollo comune di comunicazione (es. HTTP/HTTPS per il web).**

4. **Client e Server girano di solito su macchine diverse collegate in rete.**

Come il Client trova il Server: Domini e DNS

Un problema classico del web è l'identificazione dei server.

- **I server hanno indirizzi numerici univoci chiamati indirizzi IP (es. `142.250.184.142`).**
- **Gli esseri umani però trovano difficile ricordare numeri, quindi si usano i nomi di dominio (es. `www.google.com`).**

Qui entra in gioco il **DNS (Domain Name System)**, che funziona come una "rubrica telefonica di Internet":

- Traduce i nomi di dominio leggibili dall'uomo in indirizzi IP numerici, permettendo al client di raggiungere il server corretto.
  Per capire meglio come lavora il DNS vediamo il flusso di richiesta:

1. **L'utente scrive `www.google.com` nel browser (Client).**
2. **Il browser chiede al DNS: "Qual è l'indirizzo IP di `www.google.com`?"**
3. **Il DNS risponde: "L'IP è `142.250.184.142`."**
4. **Il browser invia la richiesta (es. HTTPS) all'indirizzo IP fornito (Server).**
5. **Il server riceve la richiesta, la elabora e restituisce la risposta (es. la homepage di Google).**

Indirizzo IP

Un **indirizzo IP (Internet Protocol)** è:

- **un'etichetta numerica che identifica in modo univoco un dispositivo (computer, smartphone, server, ecc.) all'interno di una rete.**
  Senza l'IP, i dispositivi non saprebbero come trovarsi e comunicare tra loro.
  Esistono 2 versioni principali:

1. IPv4

- È la versione più diffusa e conosciuta.
- **È composto da 32 bit, rappresentati come 4 numeri decimali separati da punti (*notazione dotted-quad*).**
  👉 Esempio: `192.168.1.10`

Ogni blocco (detto **ottetto**) **va da 0 a 255**.

Un indirizzo IPv4 è diviso in:

1. **Network ID** → **identifica la rete a cui appartiene il dispositivo.**
2. **Host ID** → **identifica lo specifico dispositivo all'interno di quella rete.**

📌 La separazione tra rete e host è definita dalla **Subnet Mask**.

- Esempio: `192.168.1.10` con subnet `255.255.255.0` →
  - `192.168.1` = Network ID
  - `10` = Host ID

2. IPv6

- **Nato per superare il limite degli IPv4 (che stavano esaurendo).**
- **È lungo 128 bit, quindi ha uno spazio di indirizzi praticamente illimitato.**
- **È scritto in otto gruppi di quattro cifre esadecimali, separati da due punti.**
  👉 Esempio: `2001:0db8:85a3:08d3:1319:8a2e:0370:7344`

> ⓘ **Regole di abbreviazione:**
>
> 1. Gli **zeri iniziali** si possono omettere (`0db8` → `db8`).
> 2. Una sequenza di blocchi pieni di zero si può sostituire una sola volta con `::` .
> 👉 Esempio:
> `2001:0db8:0000:0000:0000:8a2e:0370:7344` → `2001:db8::8a2e:370:7344` .
>
> Struttura IPv6:
> prefisso di rete globale + ID sottorete + ID interfaccia del dispositivo.

IP Pubblico vs IP Privato

Gli indirizzi IP possono essere di due tipi:

- **IP Pubblico** → **quello che il tuo provider (ISP) assegna per identificarti su Internet. È unico a livello mondiale.**
- **IP Privato** → **usato solo all'interno di una rete locale** (es. Wi-Fi di casa).
  Intervalli tipici: `192.168.x.x` , `10.x.x.x` .

📌 I dispositivi di casa usano IP privati e condividono un unico **IP pubblico** grazie al router, che usa il **[[Network, Transport, Session, Presentation, Application Layers#**Tecnologia NAT (Network Address Translation)**|NAT (Network Address Translation)]]**.

Domini e DNS

Un **nome di dominio,** come abbiamo già visto:

- **è un'etichetta facile da ricordare che identifica una risorsa su Internet (es.** `www.google.com` **).**
  I computer però comunicano solo con indirizzi IP → quindi serve un traduttore. E quindi entra in gioco il **DNS (Domain Name System)**: la "rubrica telefonica di Internet" che traduce i nomi di dominio negli indirizzi IP corrispondenti.

Strutture di dominio:

Quindi un dominio ha una struttura **gerarchica** che si legge da destra a sinistra.

Prendiamo come esempio questo dominio:

- `mail.google.com`

1. **TLD (Top-Level Domain)** → `.com`
   - **È la parte più a destra del dominio**
   - **Definisce la categoria più ampia del dominio**
   - Si suddividono in 2 sotto-categorie:
     - gTLD generici: `.com`, `.org`, `.net`
     - ccTLD nazionali: `.it`, `.de`, `.uk`
2. **SLD (Second-Level Domain)** → `google`
   - **È la parte centrale (** `google` **)**
   - **Questo è il nome che viene registrato ed è il cuore dell'identità del brand.**
   - **È univoco all'interno del suo TLD (esiste una sola** `google.com` **).**
3. **Subdomain (Terzo Livello)** → `mail`
   - **È la parte più a sinistra del dominio (** `mail` **).**
   - **I sottodomini vengono creati dal proprietario del dominio di secondo livello per organizzare e separare diverse sezioni o servizi del proprio sito.**
   - Alcuni esempi sono:

- `www` : **Convenzionalmente usato per il sito web principale.**
- `api` : **Spesso usato per esporre le API**
- `blog` : **Per una sezione dedicata al blog.**

Nameserver e Record DNS

Quindi se il dominio è l'indirizzo, il DNS è la "rubrica telefonica" che lo rende utilizzabile.
Questa gestione si basa su 2 componenti chiave:

1. **I Nameserver**
2. **I Record DNS**

Quando si registra un dominio, bisogna indicare i nameserver autoritativi:

- **si tratta di un server specializzato che contiene tutti i record DNS per un determinato dominio.**
- **Questi nameserver solitamente sono forniti dal tuo provider di hosting. Questi server diventano la fonte ufficiale di verità per il tuo dominio. Qualsiasi computer nel mondo che vuole sapere l'indirizzo IP del tuo sito dovrà interrogare uno di questi server.**

I Record DNS invece sono:

- **le singole voci all'interno della rubrica gestita dal nameserver.**
- **Ogni record ha un tipo specifico che definisce il tipo di informazione che fornisce.**
  I più importanti sono:
- **Record A (Address):**
  - **È il record più comune.**
  - **Associa un nome di dominio a un indirizzo IPv4.**
  - Esempio `miosito.it` <==> `88.99.100.123`

> **Significa:**
>
> "chi cerca miosito.it deve andare all'indirizzo IP `88.99.100.123`"

- **Record AAAA (Quad A):**
  - **Simile al record A, ma associa un dominio a un indirizzo IPv6**

    - Esempio: `miosito.it` AAA `2001:0db8:85a3:0000:0000:8a2e:0370:7334`
- **Record CNAME (Canonical Name):**
    - **In parole povere è alias verso un altro dominio.**
    - **In sostanza permette a un dominio di puntare a un altro dominio, invece che trovare l'indirizzo IP.**
    - Esempio: `www.miosito.it` CNAME `miosito.it`.

> **Significa:**
>
> `www.miosito.it` è solo un altro nome per `miosito.it`. Trovate l'IP di quest'ultimo".
> È utile per avere più nomi che puntano allo stesso server.

- **Record MX (Mail Exchange):**
    - **Specifica quali sono i server di posta elettronica responsabili di ricevere le email per quel dominio.**
    - Esempio: `miosito.it` MX 10 `mail.provider.com`

> **Significa:**
>
> "tutte le email indirizzate a `@miosito.it` devono essere gestite dal server `mail.provider.com`". Il numero 10 indica la priorità

- **Record TXT (Text):**
    - **Permette di associare al dominio una stringa di testo arbitrario.**
    - **È comunemente usato per scopi di verifica** (es. per dimostrare a Google o Microsoft di essere il proprietario del dominio) **o per configurazioni di sicurezza email come SPF e DKIM.**
- **Record NS (Name Server):**
    - **Indica quali sono i nameserver autoritativi per un dominio (o sottodominio), delegando di fatto la gestione.**

Processi di Risoluzione DNS

Per comprendere meglio come funzionano i processi di risoluzione DNS vediamo passo passo cosa accade quando inseriamo un nome di dominio nel browser:

1. **L'utente scrive nella barra di ricerca del browser `miosito.it`.**
2. **Il PC chiede al resolver DNS (spesso fornito dal provider).**
3. **Se non sa la risposta → interroga i Root Server.**
4. **Questi lo indirizzano al server del TLD (.it).**
5. **Il TLD dice: "Per `miosito.it`, chiedi a `ns1.tuohosting.com`."**
6. **Il nameserver autoritativo risponde con i record:**
   - `CNAME www.miosito.it` → `miosito.it`
   - `A miosito.it` → `88.99.100.123`
7. **Il resolver restituisce l'IP al tuo PC.**
8. **Il browser contatta il server all'IP `88.99.100.123`.**
   Quindi in pochi millisecondi, il DNS ha tradotto un nome facile (`www.miosito.it`) in un indirizzo numerico che i computer possono usare.

In sintesi:

- IP = numero che identifica un dispositivo in rete.
- Dominio = nome facile da ricordare per gli umani.
- DNS = sistema che traduce domini in IP.